算法设计与分析　课程报告

内存键值存储引擎中的轻量级任务调度优化

专题：任务窃取调度（Work-Stealing Scheduling）

设计思想 · 适用条件 · 实现流程 · 复杂度 · 尾延迟实验

课程名称	算法设计与分析
课题名称	内存键值存储引擎中的轻量级任务调度优化（课题 7）
专题范围	任务窃取调度的设计、实现、复杂度与实验分析
配套材料	C++17 参考实现 · 离散事件仿真器 · 复现脚本与插图
版本	v1.0 · 2026 年 11 月

摘　要

本册围绕课题 7 中的任务窃取调度展开。第 1 节给出任务模型与五条适用假设，说明窃取相对集中式 FIFO 与纯本地队列的位置；第 2 节从双端队列的所有权语义、随机受害者选择与拉取式迁移三个层面阐述设计思想，并给出理论保证与适用边界；第 3 节给出可实现的算法流程、内存序、终止检测与防抖动机制；第 4 节分析时间与空间复杂度，并建立迁移代价模型；第 5 节从排队论角度解释窃取对吞吐、平均延迟与 P99/P99.9 的影响；第 6 节用 20 线程离散事件仿真与 C++17 微基准比较集中式 FIFO、本地队列与工作窃取三种策略在均匀、偏斜和动态热点负载下的表现；第 7 节总结优缺点、适用边界与六个改进方向。

核心结论有三点。其一，窃取在偏斜与动态热点下把吞吐从 0.25 Mtask/s 恢复到 0.70 Mtask/s，P99 从约 760 ms 降到约 0.23 ms，代价是每千任务约 440 次窃取（阈值 k=2）。其二，在均匀负载下，窃取与本地队列的吞吐相同，P99 由 0.88 ms 降至 0.22 ms，但仍高于理想化的集中式 FIFO（0.15 ms）——这是“分片队列损失部分池化收益”的结构性代价，也是必须用可扩展性与局部性去交换的部分。其三，NUMA 亲和感知的窃取把跨节点执行比例从 25.4% 降到 0.1%，P99 再降 25%，说明“偷谁”与“怎么偷”同样重要。

目　录

1　问题定位与任务模型

2　设计思想

3　数据结构与实现流程

4　复杂度分析

5　对吞吐、平均延迟与尾延迟的影响

6　实验：三种负载下的三种策略

7　优缺点、适用范围与改进方向

8　小结

附录 A　核心代码摘录

附录 B　复现说明

附录 C　符号表与参考文献

1 问题定位与任务模型

课题要求至少实现三种调度算法并完成比较。任务窃取并不是与集中式 FIFO、轮询、优先级并列的又一种“排队规则”，而是在本地队列之上增加的一条再平衡规则：常态下每个核心只服务自己的队列，只有当某个核心空闲、另一个核心积压时，才发生一次跨核的任务转移。理解这一点，是理解它全部代价与收益的前提。

1.1 从每核一线程说起

每核一线程（thread-per-core）是当前内存键值引擎的主流线程模型：分片与核心一一对应，分片的内存池在属主节点上首次触碰，读写路径无锁、无共享可变状态。它把缓存一致性与 NUMA 远程访问压到最低，扩展性接近线性，却放弃了工作保持（work-conserving）性质：一个核心的队列为空时只能空转，即使另一个核心的队列已经排到几千个任务。

这种损失在三种负载下表现不同。均匀负载下，它表现为排队方差的放大：把 P 个服务台拆成 P 个独立的单台队列，在相同利用率下，平均等待从多服务台的近似零上升到单台的 $\rho/(1-\rho)$ 倍服务时间。偏斜负载下，它直接表现为木桶效应：热点分片的属主成为瓶颈，其余核心旁观，系统吞吐由最慢的那个队列决定。动态热点下，瓶颈核心还在不断变化，静态的分片归属无法跟随。

任务窃取用一条规则同时缓解这三种情况：空闲核心从其他核心的队列顶部（最早进入的一端）取走一个任务执行。绝大多数任务仍由属主执行，局部性得以保留；只有真正不均衡时才付出一次跨核转移的代价。Blumofe 与 Leiserson 证明，对严格多线程计算，其期望执行时间满足 $T_P \le T_1/P + O(T_\infty)$，这是它在语言运行时、并行框架与存储系统中被广泛采用的理论基础。

1.2 任务模型与五条假设

把引擎中的一个待执行单元抽象为六元组 $T = (op, key, arrival, service, affinity, priority)$：op 为操作类型，key 决定分片与数据位置，arrival 是入队时刻，service 是服务时间，affinity 描述任务与核心、NUMA 节点或设备的绑定关系，priority 是前台/后台优先级。窃取算法只关心两个属性：任务能否离开原核心执行（可迁移性），以及一次迁移的成本能否被随后的执行摊薄（粒度）。

表 1　任务窃取的五条适用假设

假设	含义	不满足时的后果	工程对策
A1 可迁移	任务不依赖线程本地状态	窃取后读写错误的缓冲或	把任务做成纯函数式闭

假设	含义	不满足时的后果	工程对策
	（连接、TLS、非重入会话）	连接	包；连接绑定改为请求转发
A2 粒度足够	单任务执行时间 ≥ 10 倍窃取成本	CAS 与缓存冷启动开销超过收益	先聚合为批任务；短任务按 key range 合并
A3 数据可远程访问	任务在别的核执行只会变慢，不会出错	跨节点访问导致正确性问题（如非一致映射）	副本/目录化；或只允许同节点窃取
A4 队列有界	本地队列有容量上限与背压	过载转化为无界内存增长与超时	有界队列 + 拒绝/暂停读取
A5 顺序可放松	同一 key 的并发任务有幂等或版本保护	迁移改变执行顺序导致脏写	按 key 串行化；版本号/租约

这五条不是形式化的前提，而是工程检查表。A1 与 A5 决定“能不能偷”，A2 决定“值不值得偷”，A3 决定“往哪里偷”，A4 决定“偷不过去时怎么办”。后面第 2.5 节会把它们展开成可操作的判据。

1.3 与其它五种调度算法的关系

课题列出的六种算法并不在同一个抽象层上。集中式 FIFO、轮询、优先级回答的是“取出哪一个”；本地队列与窃取回答的是“在哪个核上执行、队列如何组织”；反馈控制回答的是“参数调多少”。把它们放在一张表里，可以看清窃取的真实位置：它是本地队列的必要补丁，而不是 FIFO 的替代品。

表 2 六种调度算法的分工与关系

算法	决定什么	主要优势	主要代价	与窃取的关系
集中式 FIFO	全局出队顺序	实现简单、天然均衡、无饥饿	队头阻塞、锁争用随核数放大	可作为注入队列；窃取是其去中心化替代
轮询	任务到执行者的静态映射	公平、无共享队列	忽略任务大小与负载变化	静态分片规则；窃取补其动态失衡
优先级	多类任务的服务次序	保护前台、隔离后台	反转与饥饿	决定“能不能被偷”，与窃取正交
本地队列	任务到队列的归属	无锁、局部性好	非工作保持	窃取的载体
任务窃取	空闲时从谁那里取	均衡与局部性折中	迁移、抖动、非确定性	本地队列之上的再平衡层
反馈控制	阈值、并发度等参数	适应未知负载	振荡、滞后	为窃取阈值提供自适应

因此，一个成熟的引擎通常分层组合：入口用反馈控制做准入，全局用优先级区分前后台，每核用本地队列保持亲和，空闲时用窃取补齐不均。本册只展开最后一块，但在第 7 节会说明它与其余各层如何衔接。

1.4 设计目标与约束

窃取的设计目标不是单一吞吐，而是一组互相牵制的指标：

- **吞吐率**：所有核心的有效利用率，等价于 makespan 的倒数；偏斜负载下由最慢队列决定。
- **平均延迟**：反映常规体验，但对长尾不敏感，不能单独作为目标。
- **P99 / P99.9**：尾部体验，受排队、迁移抖动、远程访问与长任务共同影响。
- **窃取率与远程执行比例**：迁移开销与 NUMA 代价的直接度量，应作为一等指标监控。

约束来自硬件与语义两侧：核心之间的缓存行传输、跨节点互连带宽、任务的可迁移性、以及同一 key 上必须保持的顺序。算法设计能做的，是在这些约束下把“均衡的速度”和“迁移的代价”调到合适的平衡点。

2 设计思想

2.1 所有权分离与两端语义

工作窃取的载体是每个执行者私有的双端队列（deque）。队列的两端对应两种角色：所有者（owner）在底部压入新任务，也从底部弹出任务，顺序是后进先出（LIFO）；窃取者（thief）从顶部取走任务，顺序是先进先出（FIFO）。这两种顺序不是随意的选择，而是整个算法效率的基础。

所有者用 LIFO，是因为刚刚生成的任务，其输入数据大概率还在 L1/L2 缓存里，甚至就在寄存器旁边。深度优先地执行最新任务，能让缓存保持热、调用栈保持浅，这是分治类算法高效的根本原因。窃取者用 FIFO，是因为队列顶部是最早进入、尚未被执行的任务：它通常粒度最大、剩余工作量最多，偷走它对原所有者的局部性伤害最小，也最可能一次偷到“足够多”的工作，从而减少窃取次数。

两端分离还带来同步上的好处：所有者只修改 bottom，窃取者只通过 CAS 修改 top，二者在大多数时候不碰同一个缓存行。只有队列只剩一个元素时，双方才会在 CAS 上相遇——这正是算法把竞争压到最低的关键。

表 3　双端队列两端的所有权语义

维度	所有者（owner）	窃取者（thief）
操作端	底部（bottom）	顶部（top）
顺序	LIFO：优先执行最新任务	FIFO：优先取最老任务
目的	缓存热、栈浅、深度优先	粒度大、伤害小、一次偷够
同步	读 top 做边界判断，通常无需 CAS	CAS 推进 top，失败即放弃
频率	每个任务一次	仅在本队列为空时发生

2.2 随机化受害者与去中心化

窃取者不知道哪个队列有活。维护全局“最忙队列”列表会引入中心节点与额外同步，反而回到集中式队列的老路。工作窃取的做法是随机选择受害者：每次失败后换一个，直到找到或确认全局空闲。若系统中有 k 个非空队列、共 P 个执行者，单次探测命中的概率为 k/P，期望探测次数为 P/k；最坏情况（只有 1 个非空队列）为 P。由于窃取只发生在空闲时，这点探测成本被随后执行的成百上千个任务摊薄，单位任务开销接近零。

随机化还有两个副作用，都是正面的。其一，它天然打散多个窃取者同时扑向同一个热队列的概率，避免在 top 指针上形成 CAS 风暴；其二，它不依赖全局视图，系统可以任意增减线程而不改动协议。代价是收敛速度略慢，并且行为不确定——同样的输入可能产生不同的执行顺序，给调试与复现带来困难。

2.3 拉取式迁移与推送式迁移

任务迁移有两条路径。拉取式由需求方发起：空闲者主动去偷，只在真正空闲时发生，天然不会过度迁移。推送式由供给方发起：某个队列超过阈值时，主动把任务分给空闲者，均衡更快，但容易在阈值附近反复触发，形成迁移抖动。工作窃取属于拉取式，这也是它比“忙者推送”更稳的原因。

还必须把任务迁移与线程迁移区分开。线程迁移是操作系统调度器的行为：它把正在运行的线程整体搬到另一个核心，线程的栈和寄存器保持不变，但缓存、TLB、NUMA 亲和全部丢失。任务迁移保持线程不动，只移动工作本身：线程仍然绑在原来的核上，被移动的任务在新的核上执行，付出一次冷缓存与可能的远程内存访问。前者是“人搬家”，后者是“活找人”。

表 4　三种“迁移”的对比

方式	触发者	保留	丢失	适用
线程迁移（OS）	负载均衡、亲和性	线程栈、寄存器、连接	缓存、TLB、NUMA 亲和	通用计算，非绑核场景

方式	触发者	保留	丢失	适用
任务迁移（窃取）	本地队列为空	线程绑核与缓存	被迁移任务的数据局部性	每核一线程、任务可迁移
任务迁移（推送）	本地队列超阈值	全局均衡更快	可能反复迁移、抖动	突发明显、任务粒度大

2.4 理论保证

对严格多线程计算（fork-join 型任务图），工作窃取调度器的期望执行时间满足：

$$E[T_P] \le T_1 / P + O(T_\infty)$$

其中 T_1 是单处理器执行时间，T_∞ 是无穷处理器下的关键路径长度，P 是执行者数量。这个界说明：只要有足够的并行度（T_1/T_∞ 远大于 P），它就能接近线性加速；同时，采用延续窃取（continuation stealing）时，所需空间为 $O(P \cdot S_1)$，而不是朴素深度优先执行在最坏情况下的 $O(T_1)$。S_1 是单处理器执行时的栈空间上界。

需要强调这个界的三个前提：任务图是严格的（fork 与 join 配对）、受害者随机选择、窃取只发生在本地队列为空时。工程实现还要加上队列操作、CAS、缓存失效与终止检测的常数项；当任务粒度过细时，这些常数项会吞掉理论收益，这也是第 1.2 节把“粒度足够”列为假设的原因。

2.5 适用条件

把假设展开成可操作的判据，可以在设计评审时逐条检查：

表 5　任务窃取的适用条件与替代方案

条件	判据	反例	替代方案
任务可迁移	任务不读写线程本地状态；或可显式携带上下文	绑定连接的会话、thread_local 缓冲	请求转发（消息传递），不迁移
粒度足够	$E[S] \ge 10 \times C_{steal}$；经验值 20–50 μs 以上	单次哈希查找（数百 ns）	批量聚合、按 key range 切分
可切分	长任务能拆成 ≤ 尾延迟目标的子任务	一次不可分割的 100 ms compaction	按 SST/key range 切分
队列有界	本地队列有容量与背压策略	无界队列 + 过载	有界队列、暂停读取、拒绝
数据可远程	跨节点执行只增加延迟，不破坏正确性	非一致映射、绑核独占的缓冲区	同节点窃取、副本
顺序可放松	同 key 有版本号/幂等/串行化保护	无版本保护的读改写	按 key 串行、租约

反过来，以下场景不应使用窃取：强顺序事务、硬实时、任务与特定设备强绑定、以及任务短到窃取成本高于执行本身且无法批处理的情况。此时应优先考虑集中式 FIFO（顺序重要）、优先级（前后台分级）或纯本地队列（强亲和）。

2.6 在键值引擎中的落点

窃取在存储系统里最常见的用武之地不是前台点查，而是后台与批处理任务。前台请求的时延预算通常只有几百微秒，一次跨核迁移的冷缓存成本就可能吃掉预算，因此前台应保持“请求转发”而不是“任务窃取”：把 key 与操作序列化后投递给分片属主，属主执行完把结果写回完成队列。转发的是消息，不是执行上下文，既保持了数据不动，又避免了远程指针追逐。

表 6　任务窃取在键值引擎中的落点

任务类型	切分单位	典型粒度	可窃取	说明
Compaction	SST 文件 / key range	0.5–1 ms	是	切到 ≤1 ms 再入后台队列，低优先级
Flush / 落盘	immutable memtable	1–5 ms	是	按批次切分，令牌桶限流
批量回填 / 预热	key range	1–10 ms	是	粒度天然较大，适合窃取
副本同步	日志段 / 批次	0.5–2 ms	是	按批切分，避免单核长期落后
大 key 扫描	区间	1–10 ms	是	配合限流，避免打爆前台
前台点查	—	20–200 µs	否	用请求转发；不迁移执行上下文
过期扫描	分片桶	0.1–1 ms	可选	低优先级，可被偷

这里有一条与尾延迟目标直接挂钩的工程原则：**想让 P99 稳定在 1 ms，就不能有超过 1 ms 的不可分割任务**。窃取的均衡能力受最粗任务粒度限制；如果一次 compaction 是不可分割的 100 ms，那么无论怎么偷，负载都无法在毫秒级上均衡。切分粒度应当与 SLO 对齐，而不是与数据布局对齐。

3 数据结构与实现流程

3.1 双端队列的结构

Chase-Lev 双端队列用一个环形数组保存元素，用两个单调递增的整数 top 与 bottom 划分有效区间 [top, bottom)。bottom 只由所有者推进，top 只由窃取者通过 CAS 推进。数组容量取 2 的幂，下标用 i & (cap−1) 计算，避免取模。所有者压入时若发现 b − t > cap − 1，就把容量翻倍，把 [t, b) 的元素复制到新数组，再发布新的数组指针。旧数组不立即释放，而是保留到队列析构，防止仍在读取旧数组的窃取者访问已释放内存。

这种“只增不减、旧数组延迟回收”的做法牺牲了一点内存，换来了实现上的简单与正确：窃取者始终可以安全地读取它看到的那个数组，即使所有者在它读取期间完成了扩容。生产实现若在意内存，可以配合 epoch 或 hazard pointer 做安全回收，但那是另一个复杂度量级。

3.2 三条路径与内存序

队列只有三个操作，但每一个的线性化点都不同，内存序也必须与之匹配：

- **push（所有者）**：把元素写入 slot(b)，然后释放屏障，最后发布 bottom = b + 1。屏障保证元素先于新的 bottom 可见；窃取者看到 b 增加时，一定能读到已经写好的元素。
- **pop（所有者）**：先把 bottom 减一，再用 seq_cst 屏障，然后读 top。若 t < b，正常弹出；若 t == b，说明只剩一个元素，必须与窃取者用 CAS 争夺 top：CAS 成功则所有者拿走，失败则说明被偷走，恢复 bottom。
- **steal（窃取者）**：先读 top（acquire），屏障后读 bottom；若 t < b，读取 slot(t)，再 CAS 推进 top。CAS 失败说明被别的窃取者抢先，直接返回失败并换一个受害者，绝不重试同一位置。

表 7　三条路径的同步原语

操作	执行者	原子操作	内存序	失败处理
push	所有者	store bottom	relaxed + release fence	扩容（摊还 O(1)）
pop	所有者	store bottom、CAS top	relaxed + seq_cst fence	与窃取者竞争失败则返回空
steal	窃取者	load top/bottom、CAS top	acquire + seq_cst fence	换受害者，不重试
size	任意	load top/bottom	relaxed	仅用于阈值判断，允许误差

3.3 算法流程

下面给出与参考实现一致的主循环。所有者先尝试弹出自己的任务；失败后随机探测受害者；全部失败则进入退避。窃取路径单独列出，强调“失败即放弃、换目标”的语义。

```
// 所有者 + 窃取者共用的工作线程主循环（简化）
while (!terminated()) {
    Task t;
    if (deque.pop(t)) {            // ① 底部 LIFO，缓存最热
        run(t); continue;
    }
    bool got = false;
    for (int probe = 0; probe < P; ++probe) {   // ② 随机探测
        int v = random_victim();
        if (v != self && deques[v].steal(t)) {   // ③ 顶部 FIFO
            got = true; break;
        }
    }
    if (got) { run(t); continue; }
    if (all_idle()) break;         // ④ 两阶段终止检测
    backoff();                     // ⑤ 自旋 → yield → 休眠
}

// 窃取：只尝试一次 CAS，失败立即换目标
bool steal(Task& out) {
    size_t t = top.load(acquire);
    atomic_thread_fence(seq_cst);
    size_t b = bottom.load(acquire);
    if (t >= b) return false;
    out = slot(t).load(relaxed);
    return top.compare_exchange_strong(t, t + 1,
                                       seq_cst, relaxed);
}
```

这段代码有两个容易写错的点。第一，窃取失败后不能原地重试同一个 top，否则多个窃取者会在同一缓存行上反复冲突；正确做法是换一个受害者。第二，pop 在 t == b 时必须用 CAS 与窃取者争夺最后一个元素，否则会出现同一个任务被两个线程执行的严重错误。

3.4 终止检测与退避

窃取系统必须能判断“是否真的没活了”。常见做法是给每个执行者一个饥饿标志：队列为空且一次窃取失败后置位；随后检查是否所有执行者都饥饿；若是，进入两阶段确认——再扫描一次，防止恰好有新任务入队——确认后才休眠或退出。

终止检测的难点不在正确性，而在开销。空闲线程如果持续自旋窃取，会占用 CPU、污染缓存、干扰邻居，甚至让监控误判为高负载。工程上通常采用退避阶梯：先自旋若干次（几十到几百个周期），再调用 sched_yield，最后进入条件变量或 eventfd 等待；有新任务提交时通过事件唤醒。退避的每一级都要有上限，避免“睡死”导致新任务无人处理。

3.5 迁移抖动与阻尼

如果窃取过于积极，会出现迁移抖动：任务在核心之间来回移动，每次都带着冷缓存，实际执行时间反而变长，尾延迟变差。这类问题在监控上有一个典型特征——CPU 利用率很高，但吞吐不升。抑制抖动通常靠三种阻尼：

- **阈值 k**：只有受害者队列长度超过 k 时才允许窃取，避免为了一两个任务大动干戈。
- **冷却期**：一个任务被迁移后，在接下来的一段时间内不允许再次迁移，防止连环弹跳。
- **预算**：每次调度事件最多迁移 m 个任务，把迁移开销控制在总开销的固定比例以内。

还有一条设计原则：优先让空闲者来偷，而不是把忙者的任务推走。拉取式迁移由需求方发起，天然只在真正空闲时发生；推送式迁移由供给方发起，容易在阈值附近反复触发。第 6.3 节的实验说明，阈值从 1 调到 64，P99 从 0.21 ms 单调升到 4.3 ms，而窃取率只从 0.72 降到 0.29 次/任务——阈值主要买的是“少迁移”，代价是“多排队”。

3.6 工程细节

把算法写成能上生产的代码，还需要处理下面这些问题。它们不改变复杂度，却决定实测尾延迟。

表 8　实现中的常见问题与对策

问题	现象	对策
伪共享	top/bottom、统计计数落在同一缓存行，多核写导致行弹跳	按 64 字节对齐；计数器每线程本地累加
任务生命周期	窃取者仍持有指针时任务被释放	epoch/引用计数；任务池延迟回收
异常安全	任务抛异常导致 worker 退出或队列状态不一致	任务包装为 noexcept，捕获后转为错误结果
优先级反转	低优先级任务持锁，高优先级任务等待	优先级继承/天花板；或禁止跨优先级窃取
过度并置	线程数超过物理核，窃取与上下文切换叠加	绑核；线程数 = 物理核；隔离核心
唤醒风暴	一次提交唤醒所有空闲线程	只唤醒一个；批量提交；事件合并

其中伪共享最容易被忽略。队列的 top 与 bottom 如果相邻，所有者的每次 push 都会让窃取者持有的缓存行失效，反之亦然；两个变量应当各占一个缓存行（alignas(64)），或至少用填充隔开。同理，全局的 steal_success 计数器应当改为每线程本地计数，读取时再汇总。

3.7 与线程池的集成

真实的引擎不会让业务线程直接裸调双端队列，而是把它包在一个工作窃取线程池里。提交路径通常是：生产者把任务放入一个注入队列（injector），工作线程批量取走（一次 32–64 个）再压入自己的双端队列。批量注入把注入队列的争用降低一个数量级；如果注入队列用全局锁，批量是必须的，否则它又会成为新的瓶颈。

线程池还需要决定三件事：线程是否绑核（应当绑，否则窃取省下的局部性会被 OS 迁移吃掉）；任务是否允许嵌套提交（允许时要防止 join 死锁，常用做法是帮助执行被等待的子任务）；关闭时如何排空（先停止接收，再让窃取把剩余任务做完，最后统一退出）。

3.8 实现检查清单

- 双端队列的 top 与 bottom 是否分处不同缓存行？
- pop 在最后一个元素上是否用 CAS 与窃取者竞争？
- 窃取失败是否立即换受害者，而不是原地重试？
- 扩容后旧数组是否仍然有效（延迟回收）？
- 是否有两阶段终止检测与有上限的退避？
- 阈值、冷却期、迁移预算是否可配置并有监控？
- 任务是否携带优先级/可偷标记，后台任务是否被限流？
- 线程是否绑核，NUMA 节点内的窃取是否优先？

4 复杂度分析

4.1 单次操作的时间复杂度

push、pop、steal 都是 O(1)：只涉及常数次原子读写与一次 CAS。扩容需要复制 [t, b) 的元素，单次为 O(n)，但容量按倍增长，均摊到每次 push 仍是 O(1)。CAS 在无竞争时一次成功；有竞争时期望重试次数也是常数，因为失败者会换目标而不是原地自旋。

表 9 双端队列各操作的时间复杂度

操作	最好	平均/摊还	最坏	说明
push	O(1)	O(1)	O(n)（触发扩容）	扩容复制活跃元素，倍增长摊还 O(1)
pop	O(1)	O(1)	O(1)	最后一个元素需一次 CAS

操作	最好	平均/摊还	最坏	说明
steal	O(1)	O(1)	O(1)	失败即返回，不重试
size	O(1)	O(1)	O(1)	近似值，允许并发误差
终止检测	O(1)	O(P)	O(P)	扫描所有执行者的饥饿标志

4.2 窃取次数与探测成本

设一次窃取（含 CAS、缓存行转移与冷启动）的成本为 C_steal，单任务平均服务时间为 E[S]。若每执行 n 个任务发生一次窃取，则窃取带来的吞吐损失约为 C_steal / (n·E[S])。本实验中 C_steal ≈ 1 μs、E[S] ≈ 23 μs、阈值 k=2 时每任务约 0.44 次窃取，代入得约 1.9% 的开销；实测吞吐与 FIFO 相同（0.696 Mtask/s），说明这笔开销被流水线掩盖，没有成为瓶颈。

从全局看，Blumofe 与 Leiserson 的结果给出期望窃取次数为 O(P·T_∞) 量级：关键路径越长、并行度越难挖掘，需要的窃取越多；反之，只要任务图足够宽，窃取就是低频事件。随机探测的期望次数为 P/k（k 个非空队列），最坏为 P。工程上通常把单次空闲的探测上限设为 P 或 2P，失败即进入退避，避免在全局空闲时做无意义的扫描。

4.3 空间复杂度

每个执行者一个双端队列，队列中最多同时存在 Q 个待执行任务，因此队列本身的空间为 O(P·Q)。扩容时旧数组延迟回收，最坏会多占一倍左右的空间；若使用 epoch 回收，则可降到 O(P·Q) 的常数倍。

更值得关注的是调用栈。朴素的深度优先执行在串行时栈深可达 O(T_1)；采用延续窃取后，每个执行者的栈空间上界降为 O(S_1)，全局为 O(P·S_1)。这就是 Cilk 风格实现能把“空间”从与总工作量成正比降到与并行度成正比的原因，也是它在递归分治场景中比“偷子任务”更省内存的地方。

4.4 与其它算法的复杂度对比

表 10　六种调度算法的复杂度与代价对比

算法	入队	出队	空间	争用点	迁移	确定性
集中式 FIFO	O(1)	O(1)	O(Q)	全局头尾指针	每个任务都可能跨核	高
轮询	O(1)	O(1)	O(P+Q)	无	静态映射	高

算法	入队	出队	空间	争用点	迁移	确定性
优先级（堆）	O(log Q)	O(log Q)	O(Q)	堆顶	按优先级	中
本地队列	O(1)	O(1)	O(P+Q)	无	无	高
任务窃取	O(1)	O(1)	O(P·Q)	窃取时 top 的 CAS	低频、按需	低
反馈控制	O(1)	O(1)	O(P+Q)	控制器	由策略决定	中

4.5 迁移代价模型

一次跨核窃取的总代价可以拆成六项：

$$C_mig = C_dequeue + C_CAS + C_cache_cold + C_TLB + C_remote + C_sync$$

其中 C_dequeue 是出队与边界判断，C_CAS 是 top 指针的原子更新，C_cache_cold 是目标核上没有该任务数据的冷启动，C_TLB 是页表项失效，C_remote 是跨 NUMA 节点的内存访问，C_sync 是必要的屏障与唤醒。经验上，一次跨节点迁移的代价约等于数百次本地哈希查找；在本实验的模型中取 1 μs，约为单任务服务时间的 4%。

据此可以得到“值得偷”的三个条件：低频（不是每个任务都迁）、大粒度（迁过去能执行足够久）、有收益（目标核确实空闲，且迁过去后访问的数据不在原节点）。三条同时满足时，窃取的摊还成本接近零；缺一条，就会表现为吞吐不升、尾延迟变差，或者两者同时发生。

5 对吞吐、平均延迟与尾延迟的影响

5.1 排队论视角：从 M/M/1 到 M/M/P

为什么“把任务分给固定的核”会抬高尾延迟？用排队论可以看得很清楚。单服务台、利用率为 ρ 时，平均排队时间为：

$$W_q = (1/\mu) \cdot \rho / (1 - \rho)$$

当 $\rho = 0.8$ 时，$W_q = 4/\mu$，即平均要排 4 个服务时间。若 $\mu = 1/23$ μs，平均排队约 92 μs。而同样利用率下，20 个服务台共享一个队列（M/M/20），排队时间只有约 1–2 μs，因为空闲的服务台可以立即接走队首任务。分片队列把这个“池化”收益丢掉了，这是本地队列在均匀负载下 P99 仍达 0.88 ms 的根本原因。

窃取的作用，是在“保持分片”与“恢复池化”之间取折中：常态下各核服务自己的队列，保留局部性；一旦某队列超过阈值且存在空闲核，就把任务拉过去，相当于动态地把多个单台队

列粘合成一个多台队列。粘合是局部的、按需的，因此既拿到了大部分池化收益，又避免了全局队列的争用。

5.2 均匀访问

均匀负载下，各分片的到达率接近，窃取很少发生。此时工作窃取退化为“本地队列 + 偶发再平衡”：吞吐与本地队列相同，P99 由分片队列的排队决定，略高于理想的集中式 FIFO。实验数据（表 13）正是如此：三种策略的吞吐都在 0.695–0.696 Mtask/s；FIFO 的 P99 为 154.7 μs，本地队列为 880.0 μs，窃取为 217.2 μs。

窃取把 P99 从 880 μs 拉回 217 μs，靠的是把“排队最深的那个分片”的任务分给空闲核；但它没有回到 155 μs，因为阈值 k=2 意味着队列里总要留 2 个任务才触发窃取，而每个任务约 23 μs，单这一项就贡献约 46 μs；再加上迁移的冷启动与长任务（1% 的 300 μs 任务）的队头阻塞，最终落在 200 μs 量级。

5.3 偏斜访问

Zipf(1.1) 偏斜下，最热分片承担约 24% 的请求，其属主（单核）的负载约为公平份额的 5 倍。本地队列很快被填满：队列达到容量上限后新任务被丢弃，吞吐跌到 0.25 Mtask/s，10% 的请求被丢弃，完成任务的 P99 高达 759 ms——这不是“慢”，而是排在队尾的任务等了整整一轮排空。

工作窃取把热点队列的溢出部分分给 19 个相对空闲的核心。总空闲容量约为 19 × 20% = 3.8 倍单核容量，足以吸收约 3 倍于单核的溢出；因此吞吐回到 0.696 Mtask/s，丢弃率为 0，P99 为 232 μs。代价是每千任务约 437 次窃取，以及 25.4% 的跨节点执行（随机窃取时）——后者正是第 6.4 节亲和优化的目标。

5.4 动态热点

动态热点把“热点分片”每 25 ms 轮换一次。对本地队列而言，每一轮都要重新经历一次“队列堆积—排空”，而且排空速度受单核限制，上一轮的热点还没排完，下一轮的热点又来了，尾延迟持续恶化。对窃取而言，检测滞后约为“阈值 × 服务时间 + 一次窃取”，即 2 × 23 μs + 1 μs ≈ 47 μs，远小于 25 ms 的轮换周期，因此几乎可以实时跟随。实验测得动态热点下窃取的吞吐为 0.697 Mtask/s、P99 为 237 μs，与静态偏斜基本持平；本地队列则跌到 0.241 Mtask/s、12.8% 丢弃、P99 765 ms。

5.5 尾延迟的三种来源

窃取系统的尾延迟由三部分叠加：排队、迁移与远程访问。三者量级不同，治理手段也不同。

表 11 窃取系统的尾延迟来源与对策

来源	机制	典型量级	抑制手段
排队	阈值 k 内的任务等待，长任务队头阻塞	k × E[S]，数十至数百 µs	降低阈值、切分长任务、优先级
迁移	CAS、缓存冷启动、TLB 失效	约 1 µs/次，高频时累积	冷却期、迁移预算、批量窃取
远程访问	跨 NUMA 节点读写数据	本地 DRAM 的 1.4–2 倍	亲和感知窃取、副本、同节点优先
唤醒/终止	空闲线程发现新任务的延迟	数 µs 至数十 µs	事件唤醒、批量注入、减少休眠深度
抖动	任务在核间反复移动	不可预测的尖刺	阈值 + 冷却 + 预算；拉取而非推送

一个容易被忽略的事实是：**迁移本身不是尾延迟的主因，排队才是**。本实验中一次窃取只要 1 µs，而阈值每增加 1，P99 就增加约 60–70 µs。这也解释了为什么“把阈值调大以减少迁移”往往适得其反：省下的迁移成本远小于多排的队。

5.6 与优先级、隔离的耦合

窃取决定“谁来干”，优先级决定“能不能被偷”。如果后台 compaction 与前台读请求混在同一个可偷队列里，空闲核可能偷走前台任务、也可能把后台长任务偷到正在服务前台的核上，两种情况都会破坏隔离。工程上的做法是给任务打上可偷标记：前台任务不可偷（或只允许同节点偷），后台任务可偷但受限流与低优先级约束；当受害者的前台队列超过阈值时，后台任务不允许被偷走。

这与第 7.4 节的选型建议一致：窃取负责核间均衡，优先级负责前后台分级，二者正交组合。若只做窃取不做优先级，后台任务会把前台挤走；若只做优先级不做窃取，空闲核仍然帮不上忙。

6 实验：三种负载下的三种策略

6.1 实验设置

实验分两部分。第一部分是离散事件仿真，用于在可控条件下比较调度策略本身；第二部分是 C++17 微基准，用于在真实多核上验证实现的正确性与 makespan。两部分的任务粒度不同，结论互相印证而非直接相加。

表 12 仿真参数

参数	取值	说明
线程数 P	20	仿真中分为 2 个 NUMA 节点，每节点 10 核
分片数	80	= 20 × 4，分片 s 的属主为 s mod 20
服务时间	对数正态，均值 20 μs，σ = 0.5	模拟一次批量化点查/小批处理
长任务	1%，180–420 μs	模拟 compaction 切片、批量回填
到达率 λ	0.696 Mtask/s	约为容量（0.87 M/s）的 80%
队列容量	32768 / 队列	满则丢弃，模拟有界队列的背压
窃取阈值 k	2（扫描实验 1–64）	受害者队列长度 > k 才允许窃取
窃取成本	1 μs	CAS + 缓存冷启动
FIFO 开销	0.3 μs/任务	全局队列的争用近似
仿真时长	0.5 s（约 34.8 万任务）	排空阶段计入 makespan

主实验刻意关闭了 NUMA 远程惩罚与锁排队，以隔离“调度策略”这一个变量。因此 FIFO 的结果应理解为它的理论上界：没有争用、没有远程访问。第 6.4 节单独打开远程惩罚，比较随机窃取与亲和感知窃取；锁排队的影响在第 6.6 节作为有效性威胁讨论。

若要在真实引擎上复测，建议以前台 YCSB 的 workload A（读写各半）、B（读多写少）、C（纯读）作为请求流，用 requestdistribution=zipfian 与 hotspotdatafraction 复现偏斜与动态热点；把 RocksDB 的 compaction、flush 作为可窃取的后台任务注入同一个线程池。除 YCSB 的 Throughput 与 Latency 外，还应单独采集 P99.9、每核队列长度、窃取成功/失败次数，以及 numastat 的远程分配比例——后者是判断“亲和是否被窃取破坏”的直接证据。

6.2 主结果

表 13 主实验结果（吞吐单位 Mtask/s，延迟单位 μs；k = 2）

负载	策略	吞吐	丢弃率	平均	P99	P99.9	窃取/千任务
均匀	FIFO	0.696	0	25.0	154.7	416.2	—
均匀	本地队列	0.695	0	154.2	880.0	1358.5	—
均匀	工作窃取	0.696	0	35.8	217.2	426.9	296.8
偏斜	FIFO	0.696	0	25.0	172.1	417.5	—
偏斜	本地队列	0.250	10.0%	125251.8	759390.3	760862.9	—
偏斜	工作窃取	0.696	0	35.3	232.5	429.2	436.9
动态	FIFO	0.697	0	25.1	186.4	417.6	—
动态	本地队列	0.241	12.8%	125219.8	764707.5	765622.7	—
动态	工作窃取	0.697	0	35.5	237.4	428.7	458.6

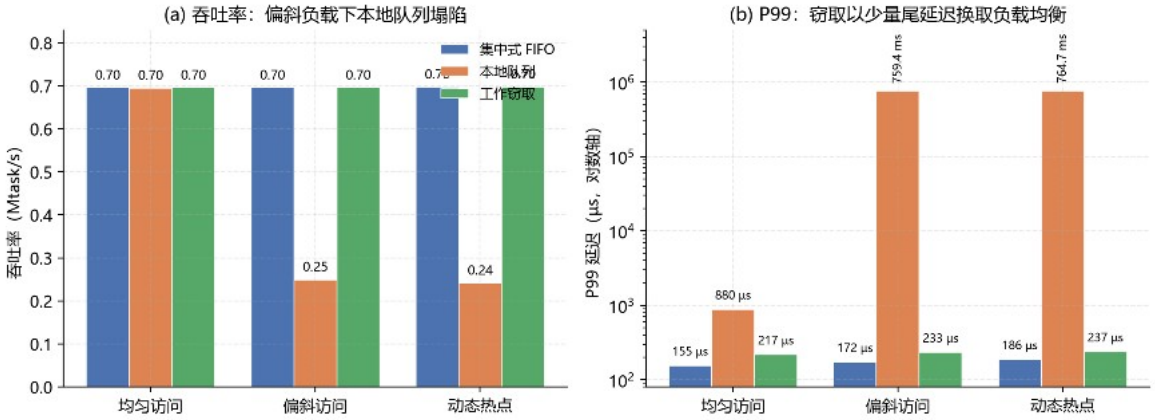


图 1　三种负载下的吞吐率与 P99：窃取在偏斜/动态热点下恢复吞吐，在均匀负载下接近 FIFO

先看吞吐。均匀负载下三种策略几乎相同（0.695–0.696），说明窃取在没有失衡时不会拖慢系统；偏斜与动态热点下，本地队列跌到 0.25/0.24，而窃取保持在 0.696/0.697，与 FIFO 持平。这是因为热点分片的溢出量（约 3 倍单核容量）小于其余 19 核的空闲容量（约 3.8 倍），窃取足以吸收。

再看尾延迟。均匀负载下 FIFO 的 P99 最低（154.7 μs），窃取为 217.2 μs，本地队列为 880.0 μs。这正是第 5.1 节预言的“池化损失”：FIFO 有 20 个服务台共享一个队列，窃取只有 1 个服务台加按需扩容，本地队列则是彻底的单台。偏斜负载下排序反转：FIFO 172 μs、窃取 232 μs、本地队列 759 ms。本地队列的 759 ms 不是服务慢，而是队列被塞满后，排在队尾的请求要等一整轮排空。

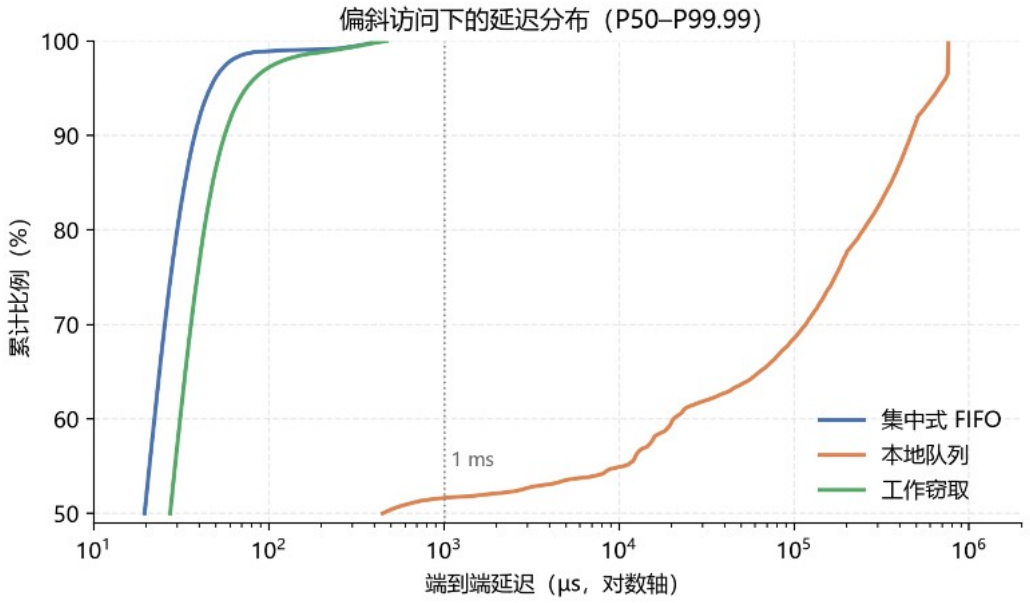


图 2　偏斜访问下的延迟分布：本地队列在 99% 之后迅速抬升到数百毫秒，窃取与 FIFO 保持在同一量级

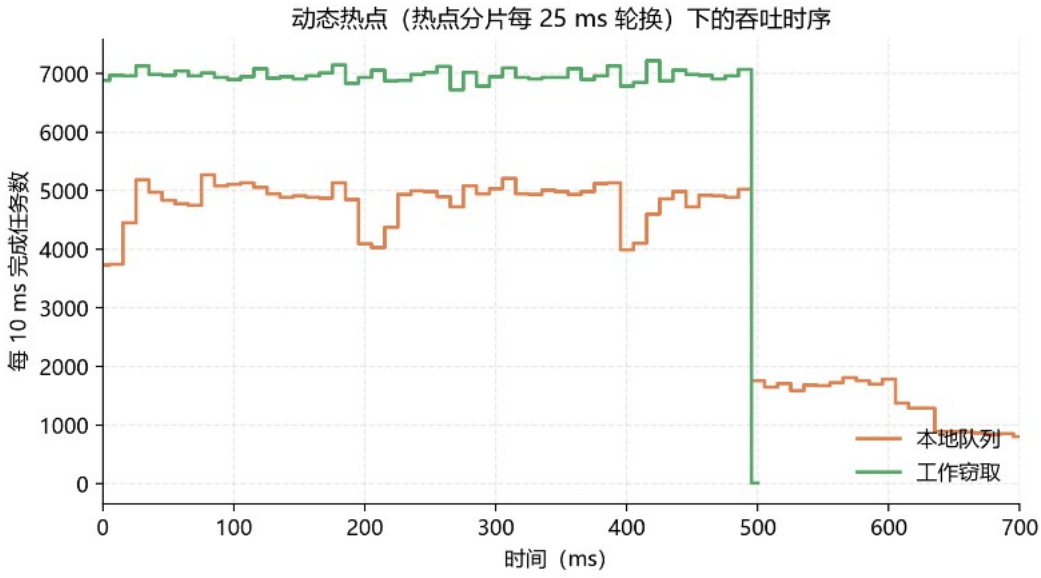


图 3　动态热点下的吞吐时序：本地队列随热点轮换反复塌陷，窃取保持平稳（每 10 ms 统计）

6.3　阈值的影响

表 14　窃取阈值扫描（偏斜访问）

阈值 k	P99（μs）	P99.9（μs）	窃取/千任务	每任务窃取率
1	211.9	421.1	715.1	0.72
2	232.5	429.2	436.9	0.44
4	295.8	451.9	342.2	0.34
8	479.8	797.0	308.0	0.31
16	1017.9	2051.7	292.2	0.29

阈值 k	P99（μs）	P99.9（μs）	窃取/千任务	每任务窃取率
32	2208.4	5199.5	286.9	0.29
64	4304.0	14640.8	285.2	0.29

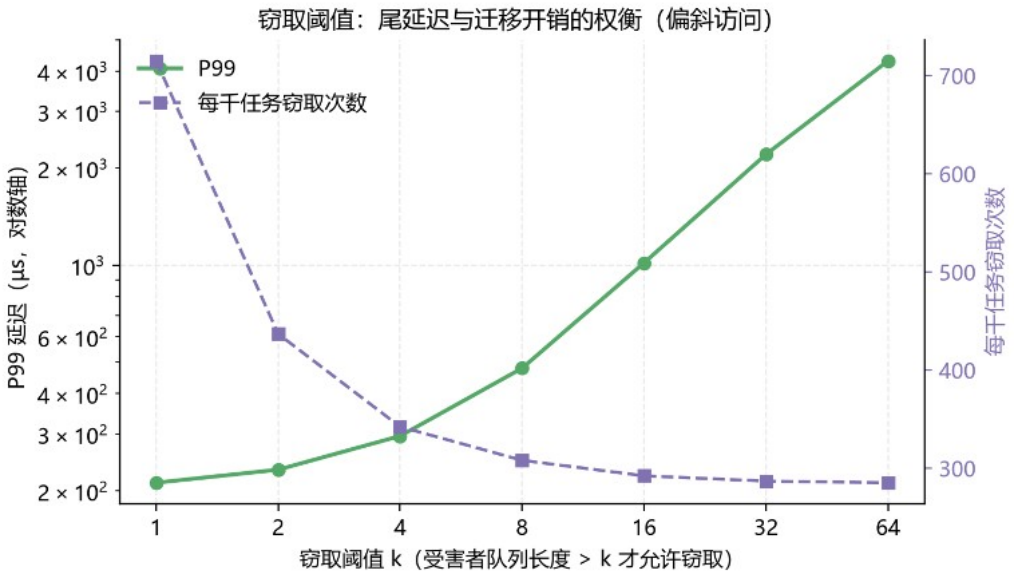


图 4　阈值从 1 增到 64：P99 近似线性上升，窃取率下降但很快饱和

阈值把“迁移频率”和“排队长度”放在一个旋钮的两端。k 从 1 增到 64，窃取率从 0.72 次/任务降到 0.29，但 P99 从 0.21 ms 升到 4.3 ms，近似线性（斜率约 60–70 μs/阈值）。这是因为受害者队列里始终积压着约 k 个任务，每个约 23 μs，再叠加长任务与迁移的方差。窃取率在 $k \geq 8$ 后基本饱和（0.31→0.29），说明此时再增大阈值只是让任务多排队，并不能显著减少迁移——**阈值的主要收益在前几档，过了拐点就是纯损失**。

对本实验的参数，k = 1–2 是延迟优先配置，k = 4–8 是吞吐/迁移折中配置；若 SLO 是 1 ms，k 不应超过 8。这也给出一个可操作的初值公式：$k \approx \text{ceil}(T_slo / E[S])$，再在运行期用反馈控制微调。

6.4　NUMA 亲和感知

随机选择受害者会把任务偷到另一个 NUMA 节点，执行时访问远程内存。模型给跨节点执行加 35% 的服务时间惩罚，比较两种受害者选择策略：随机窃取，以及优先同节点、同节点无活时才跨节点的亲和感知窃取。

表 15　随机窃取与亲和感知窃取的对比

负载	策略	P99（μs）	远程执行比例	吞吐（Mtask/s）
均匀	随机窃取	243.4	15.9%	0.695

负载	策略	P99（μs）	远程执行比例	吞吐（Mtask/s）
均匀	亲和感知	199.4	0.1%	0.696
偏斜	随机窃取	275.9	25.4%	0.696
偏斜	亲和感知	206.9	0.0%	0.696

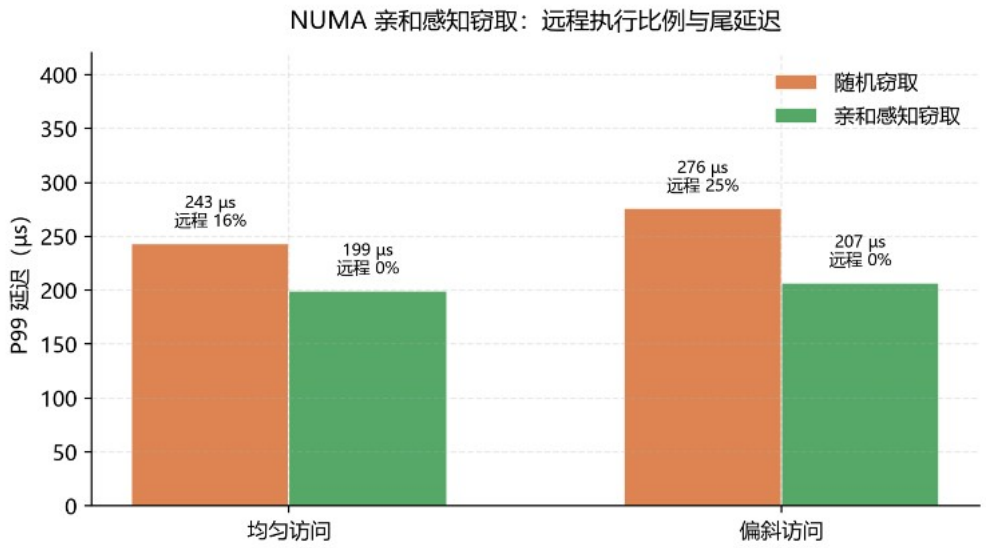


图 5　亲和感知窃取把跨节点执行比例压到接近零，P99 同步下降约 25%

结果很直接：偏斜负载下随机窃取有 25.4% 的任务在异地执行，亲和感知把它降到 0.0%，P99 从 275.9 μs 降到 206.9 μs（−25%）。均匀负载下也有 15.9% → 0.1% 的改善。原因有两个：均匀负载下窃取虽少，但每次随机偷有一半概率跨节点；偏斜负载下热点集中在单个节点，同节点的 9 个核先吸收，跨节点只在同节点也忙时才发生。

这说明“偷谁”与“怎么偷”同样重要。一个实用的分级策略是：先探测同 LLC 的队列，再探测同节点的队列，最后才跨节点；跨节点窃取设置更长的冷却期与更低的预算。本实验的亲和策略只做了第二、三级，已经拿到大部分收益。

6.5　C++ 微基准

仿真之外，用 C++17 在真实的 20 个逻辑处理器上跑一组对照：200 万个任务，80 个分片，每个任务做 512 次相关哈希探测（约数微秒），1% 的长任务做 16384 次。三种策略处理同一批任务，从同一 barrier 起跑，比较 makespan 与吞吐。任务在计时前预生成，避免把构造开销混入结果；工作窃取的双端队列采用第 3 节的 Chase-Lev 实现，正确性由独立压力测试验证（20 万项恰好消费一次）。

表 16　20 线程 C++ 微基准（2,000,000 任务；重复运行波动约 5%–8%）

负载	策略	makespan（ms）	吞吐（Mtask/s）	相对窃取
均匀	集中式 FIFO	680.1	2.941	0.64×
均匀	本地队列	483.2	4.139	0.90×
均匀	工作窃取	436.9	4.578	1.00×
偏斜	集中式 FIFO	541.5	3.693	1.32×
偏斜	本地队列	1258.0	1.585	3.07×
偏斜	工作窃取	410.4	4.874	1.00×

均匀负载下窃取的 makespan 比 FIFO 短 36%，比本地队列短 10%。FIFO 慢的原因不是排队，而是局部性：全局游标把任务随机分给任意核，执行时访问的分片表不在本地缓存；同时游标的原子操作在多核上形成缓存行争用。窃取让属主执行自己的 4 个分片，只有少量任务被偷，缓存命中率更高。本地队列在均匀负载下只慢 10%，说明“没有窃取”本身不是问题，问题在于长任务造成的临时失衡。

偏斜负载下差距拉大：本地队列的 makespan 是窃取的 3.07 倍（1258 ms 对 410 ms），FIFO 也比窃取慢 32%。FIFO 在这里的劣势同样来自局部性——它能把任务均衡地分给 20 个核，但每个核都要访问不在本地的分片表。这个结果与仿真互相印证：**窃取的价值不仅在均衡，还在“让大多数任务在数据所在的核心执行”**。

6.6 结果讨论与有效性威胁

上述结论有明确的适用边界，需要如实说明：

- **FIFO 被理想化**。仿真关闭了锁排队，C++ 基准用原子游标而非互斥队列；真实的全局队列在 32 核以上会因缓存行弹跳与唤醒开销出现负扩展。因此“FIFO 的 P99 更低”只在低争用、任务较粗的条件下成立。
- **任务粒度偏大**。仿真单任务 20 µs、微基准约数微秒，属于“批任务”量级；若单任务是数百纳秒的哈希查找，窃取与队列开销占比会显著上升，需要先聚合。
- **单种子与样本量**。主实验使用固定种子（20261104），34.8 万样本对 P99.9 约有 350 个观测点；以种子 7 复核时，均匀负载 P99 为 154.7→159.4 µs、偏斜下窃取为 232.5→233.8 µs、本地队列为 759→755 ms，偏差均在 5% 以内，但极端尾部的估计仍有不确定性。
- **丢弃模型**。本地队列在偏斜下丢弃 10–13% 的请求，这部分延迟未计入分位数；若改为无界队列，P99 会更高、吞吐更低，结论方向不变但数值不可直接比较。
- **硬件差异**。C++ 基准运行在 20 个逻辑处理器上，未绑核、未隔离，受 SMT 邻居与后台进程影响；仿真中的双节点拓扑是建模设定，不代表本机实测 NUMA 行为。

因此，本册把仿真结论定位为“机制验证”而非“性能承诺”：它说明在给定假设下，窃取如何改变排队结构；真实系统还需要用 YCSB + perf 在目标平台上复测，并按第 6.3 节的方法重新标定阈值。

7 优缺点、适用范围与改进方向

7.1 优点

- **兼顾均衡与局部性**：绝大多数任务由属主执行，只有真正失衡时才跨核；这是它相对集中式队列的核心优势。
- **去中心化、可扩展**：没有全局队列与全局锁，窃取只碰受害者队列的 top，核数增加时不会出现单点瓶颈。
- **理论保证**：对严格计算有 $T_P \le T_1/P + O(T_\infty)$ 的期望界，空间可做到 $O(P \cdot S_1)$。
- **对不规则负载鲁棒**：偏斜、动态热点、递归分治、粒度不均的扫描任务都能受益；本实验中吞吐恢复 2.8 倍。
- **摊还成本低**：窃取是低频事件，单次约 1 μs，被随后执行的成百上千个任务摊薄。

7.2 缺点

- **非确定性**：同样的输入可能产生不同的执行顺序，给调试、复现与审计带来困难。
- **迁移抖动**：阈值过低或负载快速变化时，任务在核间弹跳，CPU 利用率高但吞吐不升。
- **NUMA 不友好（默认实现）**：随机受害者会把任务偷到远程节点；本实验中 25.4% 的执行是远程的。
- **终止检测复杂**：需要饥饿标志、两阶段确认与退避，写错会导致空转或提前退出。
- **对强亲和任务无效**：绑定设备、连接或线程本地状态的任务不能迁移，只能转发请求。

7.3 改进方向

结合本册的实验，下面六个方向按收益从高到低排列：

1. **亲和感知窃取（已验证）**。受害者选择按“同 LLC → 同节点 → 跨节点”分级，跨节点设置更长冷却期。本实验把远程执行从 25.4% 降到 0.0%，P99 降低 25%。
2. **自适应阈值**。以 $k \approx \mathrm{ceil}(T_{slo} / E[S])$ 为初值，用 AIMD 根据 P99 误差调整：连续三个窗口超过目标 1.2 倍则 k 减半，连续十个窗口低于 0.8 倍则 k 加一。阈值的主要收益在前几档，因此调整应“快降慢升”。

3. **批量窃取（偷一半）**。任务很短（< 数 μs）时，一次只偷一个会让 CAS 与唤醒成本超过执行；一次取走队列的一半，可把窃取次数降一个数量级，代价是可能造成新的不均。
4. **延续窃取（continuation stealing）**。任务分裂时，当前线程继续执行一个分支，把“延续”留给窃取者。它把串行执行时的栈深从 O(T_1) 降到 O(P·S_1)，适合递归分治与深度很大的扫描。
5. **优先级感知与可偷标记**。前台任务只允许同节点窃取或禁止窃取；后台任务可偷但受令牌桶限制；受害者前台队列过长时，后台任务不允许被偷走。这样窃取不会破坏前后台隔离。
6. **与分片迁移协同**。秒级以上的持续热点，应该迁移分片所有权（连同内存与路由），而不是持续偷任务；毫秒级的瞬时热点用窃取。两种机制的时间尺度不同：窃取管“这一瞬”，迁移管“这一段时间”。

此外还有两个工程性改进：把窃取统计做成每线程本地计数，避免全局计数器成为新的争用点；把“窃取失败”的退避与提交唤醒合并成一个事件机制，减少空闲线程的无效扫描。

7.4 选型建议

表 17 按场景选择调度策略

场景	建议	理由
任务同质、核数 ≤ 8、顺序重要	集中式 FIFO	实现简单、无饥饿、顺序确定；争用尚可接受
强亲和、任务不可迁移	本地队列 + 请求转发	迁移会破坏正确性或局部性
前后台混合、SLO 严格	优先级 + 窃取（后台可偷）	隔离长任务，同时利用空闲核
偏斜/动态热点、任务 ≥ 20 μs	亲和感知窃取 + 自适应阈值	本实验验证的组合
递归分治、栈空间敏感	延续窃取	空间 O(P·S_1)，避免栈溢出
超短任务（< 1 μs）	先聚合再窃取	否则 CAS 与唤醒成本超过执行

8 小结

任务窃取是“本地队列”的必要补丁：它用一条“空闲者从受害者顶部取任务”的规则，把分片队列在失衡时重新粘合成一个多服务台系统，同时保留常态下的数据局部性。它的设计核心是所有权分离（owner LIFO / thief FIFO）、随机化受害者与拉取式迁移；它的代价是迁移、抖动、NUMA 远程访问与非确定性。

本册用 20 线程仿真与 C++ 微基准验证了三点：偏斜与动态热点下，窃取把吞吐从 0.25 Mtask/s 恢复到 0.70 Mtask/s，P99 从约 760 ms 降到约 0.23 ms；均匀负载下，它以约 40% 的 P99

代价换取可扩展性与局部性，仍显著优于纯本地队列；亲和感知窃取把跨节点执行从 25.4% 降到 0.1%，P99 再降 25%。阈值实验进一步说明，尾延迟主要由“队列里排了多少任务”决定，而不是“迁移了多少次”——这为参数选择提供了直接依据。

把窃取放回课题的六种算法中，它既不是万能的，也不是可有可无的：它是每核一线程架构在“均衡”这一维上的标准答案，但必须与优先级、限流、分片迁移和反馈控制组合，才能同时守住吞吐与尾延迟。

附录 A　核心代码摘录

以下摘自配套的 C++17 参考实现 `ws_deque.hpp`，展示 Chase-Lev 双端队列的所有者路径与窃取路径。完整代码、压力测试与微基准见 `code/` 目录。

```
// 所有者：底部压入（仅 owner 线程调用）
void push(T v) {
    size_t b = bottom_.load(relaxed);
    size_t t = top_.load(acquire);
    Array* a = array_.load(relaxed);
    if (b - t > a->cap - 1) a = grow(a, b, t);   // 满则翻倍
    a->at(b).store(std::move(v), relaxed);
    atomic_thread_fence(release);                 // 元素先于 bottom 可见
    bottom_.store(b + 1, relaxed);
}

// 所有者：底部弹出；只剩一个时与窃取者 CAS 竞争
bool pop(T& out) {
    size_t b = bottom_.load(relaxed);
    size_t t = top_.load(relaxed);
    if (b <= t) return false;
    --b;
    Array* a = array_.load(relaxed);
    bottom_.store(b, relaxed);
    atomic_thread_fence(seq_cst);
    t = top_.load(relaxed);
    if (t > b) { bottom_.store(b + 1, relaxed); return false; }
    out = a->at(b).load(relaxed);
    if (t == b &&
        !top_.compare_exchange_strong(t, t + 1, seq_cst, relaxed)) {
        bottom_.store(b + 1, relaxed);            // 被偷走，恢复
        return false;
    }
    return true;
}

// 窃取者：顶部取最老任务；失败即放弃
bool steal(T& out) {
    size_t t = top_.load(acquire);
    atomic_thread_fence(seq_cst);
    size_t b = bottom_.load(acquire);
    if (t >= b) return false;
    Array* a = array_.load(acquire);
    out = a->at(t).load(relaxed);
    return top_.compare_exchange_strong(t, t + 1, seq_cst, relaxed);
}
```

压力测试（`bench_ws --selftest`）让 1 个所有者与 4 个窃取者并发处理 20 万项，校验每项恰好被消费一次；微基准（`bench_ws 2000000 20 skew`）在 20 线程上比较三种策略的 makespan。两者均随附源码与运行命令。

附录 B　复现说明

目录结构与运行方式如下（Windows / Linux 均可）：

```
课题7_任务窃取/
├── 任务窃取调度_课题7.docx        本报告
├── code/
│   ├── ws_deque.hpp               Chase-Lev 双端队列（C++17）
│   ├── bench_ws.cpp               20 线程微基准 + 正确性自测
│   ├── sim_ws.py                  离散事件仿真（三种策略、三种负载）
│   └── plot_results.py            由结果 JSON 生成插图
├── results/                       sim_main.json / sim_sweep.json / sim_affinity.json
└── fig/                           报告插图（PNG）

# C++ 参考实现
g++ -O2 -std=c++17 -pthread code/bench_ws.cpp -o bench_ws
./bench_ws --selftest
./bench_ws 2000000 20 skew
./bench_ws 2000000 20 uniform

# 仿真与插图
python code/sim_ws.py --out results/sim_main.json
python code/plot_results.py
```

仿真的随机种子固定为 20261104，所有延迟数字均可在 `results/*.json` 中复核。C++ 微基准的绝对数值随机器、编译器与后台负载浮动，比较时应以同一轮内的相对值为准。

附录 C　符号表与参考文献

表 18　符号表

符号	含义	符号	含义
P	执行者/工作线程数	T_1	单处理器执行时间
T_∞	无穷处理器下的关键路径	S_1	单处理器栈空间上界
E[S]	单任务平均服务时间	C_{steal}	一次窃取的成本
k	窃取阈值	ρ	服务台利用率
Q	单队列容量	λ / μ	到达率 / 服务率

参考文献：

- Blumofe R D, Leiserson C E. Scheduling Multithreaded Computations by Work Stealing. Journal of the ACM, 1999, 46(5): 720–748.
- Lé N M, Pop A, Cohen A, Zappa Nardelli F. Correct and Efficient Work-Stealing for Weak Memory Models. PPoPP 2013.
- Cooper B F, Silberstein A, Tam E, et al. Benchmarking Cloud Serving Systems with YCSB. SoCC 2010.
- Dean J, Barroso L A. The Tail at Scale. Communications of the ACM, 2013, 56(2): 74–80.
- Herlihy M, Shavit N. The Art of Multiprocessor Programming. 2nd ed. Morgan Kaufmann, 2020.
- Gunther N J. Guerrilla Capacity Planning. Springer, 2007.